

本节内容

队列

基本概念

知识总览

队列(Queue)

定义

基本操作

懶 懶 der ...



注：数据结构三要素——逻辑结构、数据的运算、存储结构（物理结构）

存储结构不同，运算的实现方式不同

队列的定义

线性表是具有相同数据类型的 n ($n \geq 0$) 个数据元素的有限序列, 其中 n 为表长, 当 $n = 0$ 时线性表是一个空表。若用 L 命名线性表, 则其一般表示为

$$L = (a_1, a_2, \dots, a_i, a_{i+1}, \dots, a_n)$$

进栈

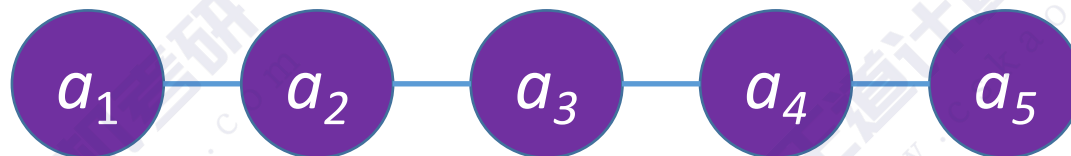
出栈

栈 (Stack)是只允许在一端进行插入或删除操作的**线性表**

队列 (Queue)是只允许在一端进行插入, 在另一端删除的**线性表**

入队

出队



队列的定义

队列（Queue）是只允许在一端进行插入，在另一端删除的线性表

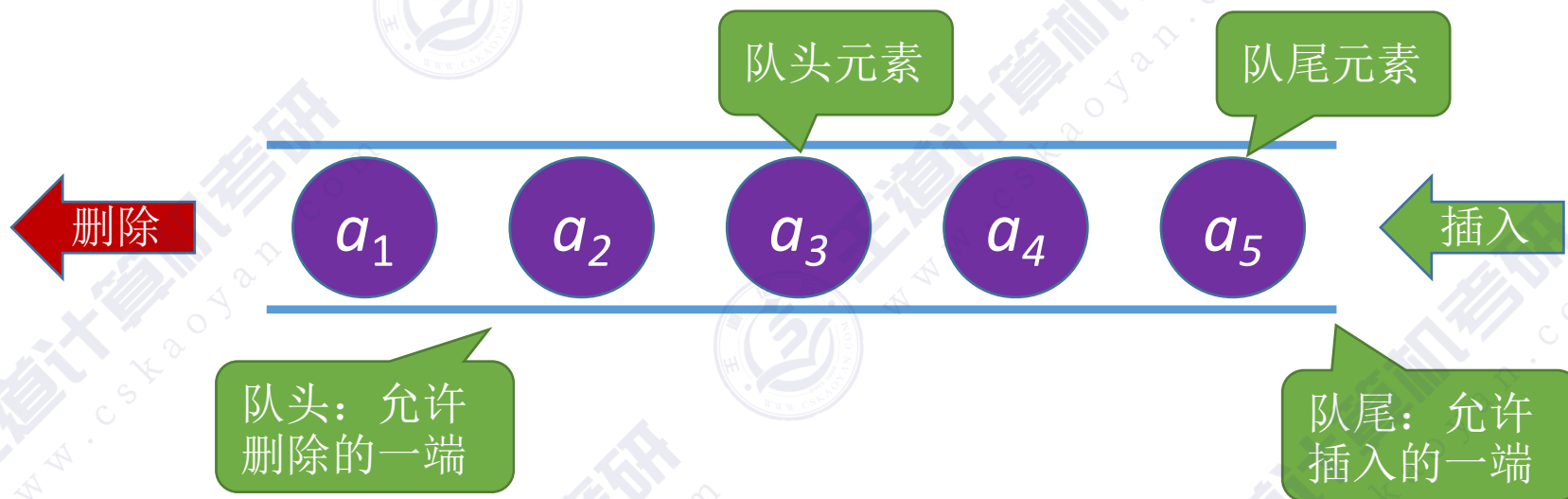


特点：先进入队列的元素先出队

队列的定义

队列（Queue）是只允许在一端进行插入，在另一端删除的线性表

重要术语：队头、队尾、空队列



队列的特点：先进先出
First In First Out (FIFO)

队列的基本操作

InitQueue(&Q): 初始化队列，构造一个空队列Q。

DestroyQueue(&Q): 销毁队列。销毁并释放队列Q所占用的内存空间。

创、销

删除队头元素

EnQueue(&Q,x): 入队，若队列Q未满，将x加入，使之成为新的队尾。

DeQueue(&Q,&x): 出队，若队列Q非空，删除队头元素，并用x返回。

增、删

不删除队头元素

GetHead(Q,&x): 读队头元素，若队列Q非空，则将队头元素赋值给x。

查：队列的使用场景中大多只访问队头元素

其他常用操作：

QueueEmpty(Q): 判队列空，若队列Q为空返回true，否则返回false。

懶 懶 der ...



知识回顾与重要考点

队列

定义

一种操作受限的线性表，只能在队尾插入、在队头删除

特性：先进先出 (FIFO)

术语：队头、队尾、空队列、队头元素、队尾元素

基本操作

创、销

增、删（入队、出队，只能在规定的—端进行）

查（获得队头元素，但不删除）

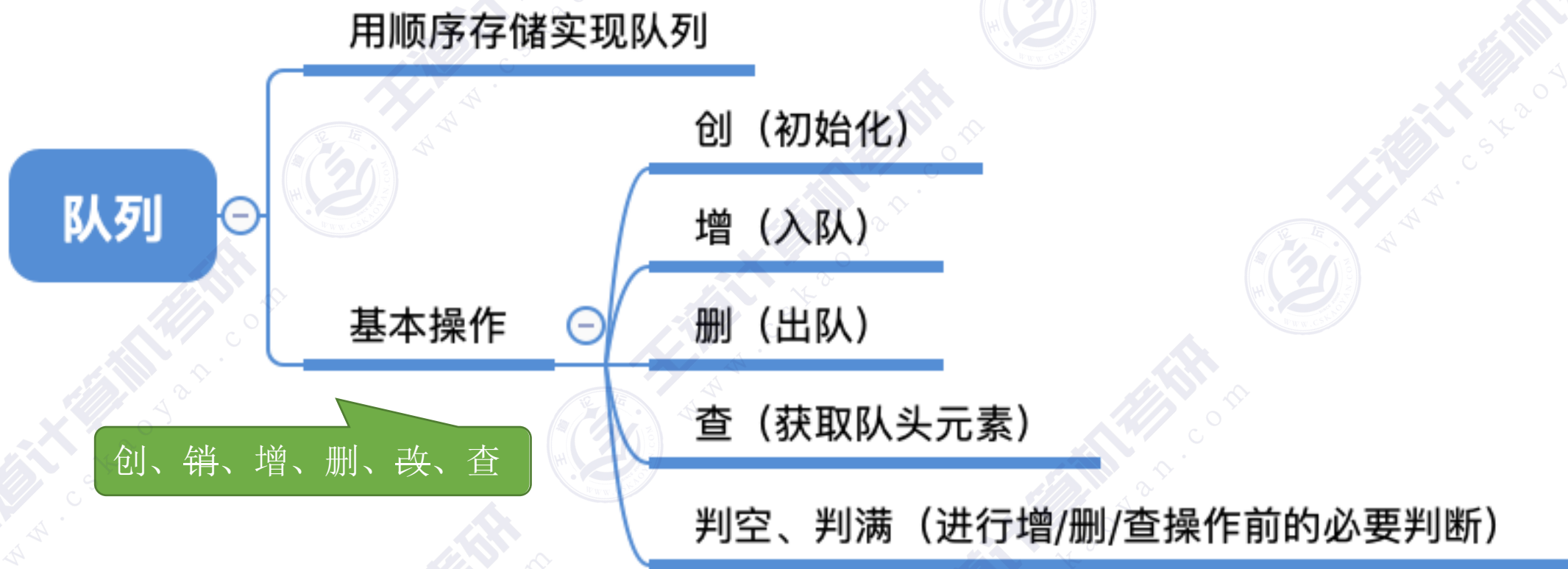
判空

本节内容

队列

顺序实现

知识总览



队列的顺序实现

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int front, rear;
} SqQueue;
```

Sq: sequence —— 顺序

rear

英 [nə(r)] 美 [rɪr]

n. 后面; 后方部队; 屁股
adj. 后方的, 后面的; 背面的

```
void testQueue(){
    → SqQueue Q; //声明一个队列 (顺序存储)
    //...后续操作...
}
```

//定义队列中元素的最大个数

//用静态数组存放队列元素
//队头指针和队尾指针

连续的存储空间, 大小
 $\text{MaxSize} * \text{sizeof}(\text{ElemType})$

front

英 [frʌnt] 美 [frʌnt]

n. 前面; 正面; 前线
vt. 面对; 朝向; 对付
vi. 朝向
adj. 前面的; 正面的

指向队尾元素的
后一个位置
(下一个应该
插入的位置)

rear

指向队
头元素

front

内存

0

front

5

rear

data[9]

data[8]

data[7]

data[6]

data[5]

e

d

c

b

a

初始化操作

```
#define MaxSize 10
```

```
typedef struct{
```

```
ElemType data[MaxSize];
```

```
int front, rear;
```

```
} SqQueue;
```

```
//初始化队列
```

```
void InitQueue(SqQueue &Q){
```

```
    //初始时 队头、队尾指针指向0
```

```
    → Q.rear=Q.front=0;
```

```
}
```

```
void testQueue(){
```

```
    //声明一个队列 (顺序存储)
```

```
    SqQueue Q;
```

```
    → InitQueue(Q);
```

```
    //...后续操作...
```

```
}
```

```
//定义队列中元素的最大个数
```

```
//用静态数组存放队列元素
```

```
//队头指针和队尾指针
```

```
//判断队列是否为空
```

```
bool QueueEmpty(SqQueue Q){
```

```
    if(Q.rear==Q.front) //队空条件
```

```
        return true;
```

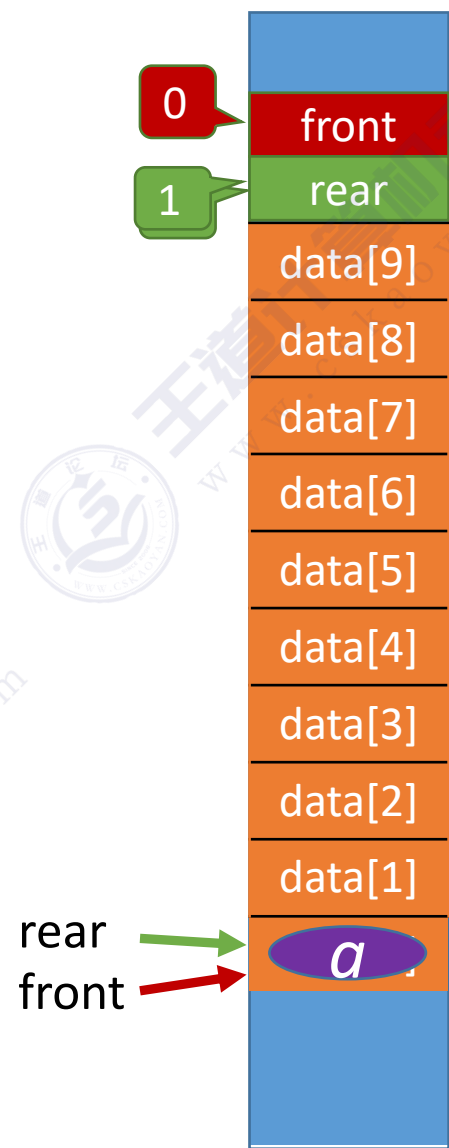
```
    else
```

```
        return false;
```

```
}
```

增删改查

内存



入队操作

只能从队尾入队（插入）

```
#define MaxSize 10
```

```
typedef struct{
```

```
ElemType data[MaxSize];
```

```
int front, rear;
```

```
} SqQueue;
```

//定义队列中元素的最大个数

//用静态数组存放队列元素

//队头指针和队尾指针

队列已满的条件：
 $\text{rear} == \text{MaxSize} ???$

//入队

```
bool EnQueue(SqQueue &Q, ElemType x){
```

```
    if(队列已满)
```

```
        return false;
```

//队满则报错

```
    Q.data[Q.rear]=x;
```

//将x插入队尾

```
    Q.rear=Q.rear+1;
```

//队尾指针后移

```
    return true;
```

```
}
```

内存

3

front

rear

10

j

i

h

g

f

e

d

c

b

a

rear

front

入队操作

只能从队尾入队（插入）

```
#define MaxSize 10
```

```
typedef struct{
```

```
ElemType data[MaxSize];
```

```
int front, rear;
```

```
} SqQueue;
```

//定义队列中元素的最大个数

//用静态数组存放队列元素

//队头指针和队尾指针

//入队

```
bool EnQueue(SqQueue &Q, ElemType x){
```

```
if(队列已满)
```

```
return false;
```

//队满则报错

```
Q.data[Q.rear]=x;
```

//新元素插入队尾

```
Q.rear=(Q.rear+1)%MaxSize;
```

//队尾指针加1取模

```
return true;
```

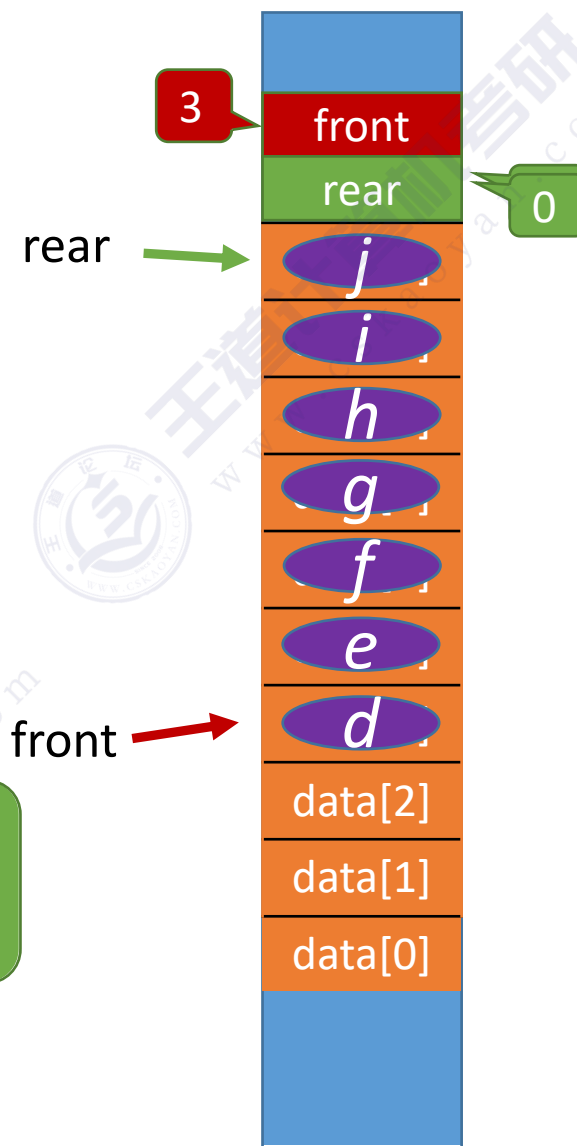
```
}
```

{0, 1, 2, ..., MaxSize-1}
将存储空间在逻辑上
变成了“环状”

模运算将无限的整数
域映射到有限的整数
集合 {0, 1, 2, ..., b-1} 上

跨考Tips: 取模运算, 即取余运算。两个整数 a, b , $a \% b == a$ 除以 b 的余数
在《数论》中, 通常表示为 $a \text{ MOD } b$

内存



循环队列

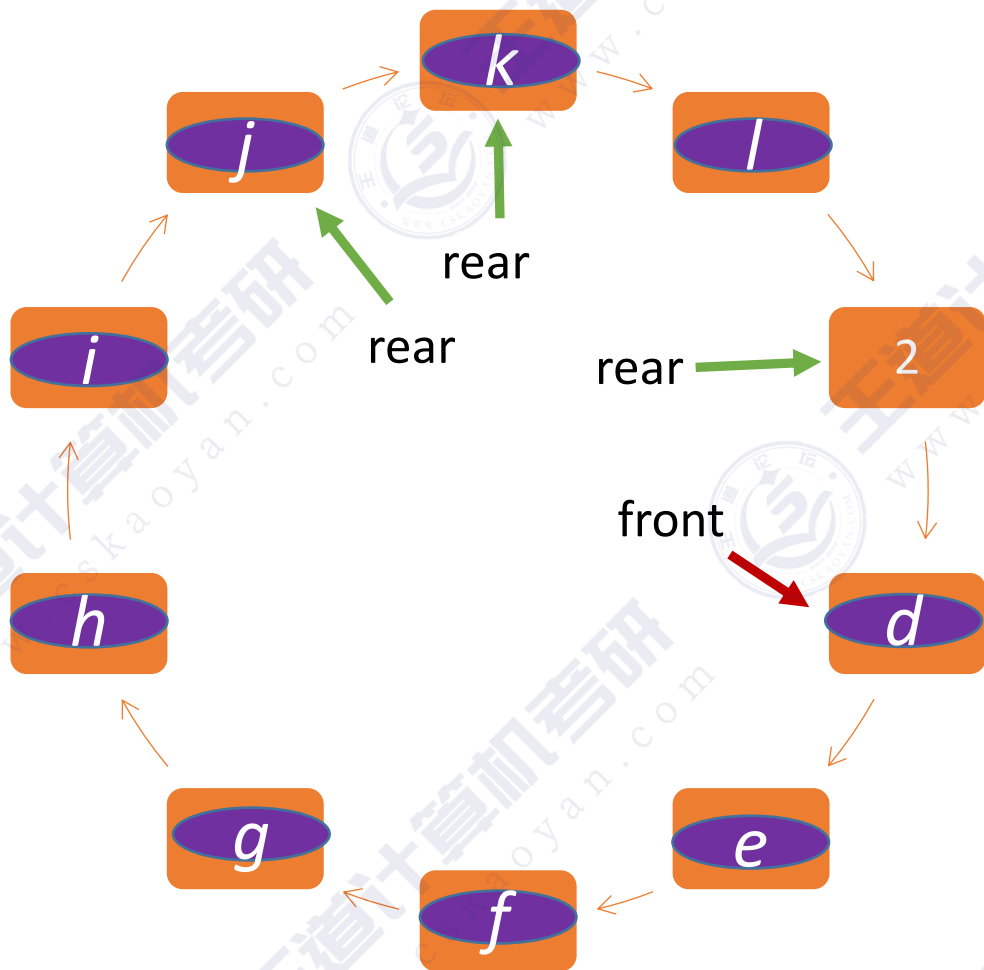
用模运算将存储空间在逻辑上变成了“环状”

➡ $Q.data[Q.rear]=x;$

//新元素插入队尾

➡ $Q.rear=(Q.rear+1)\%MaxSize;$

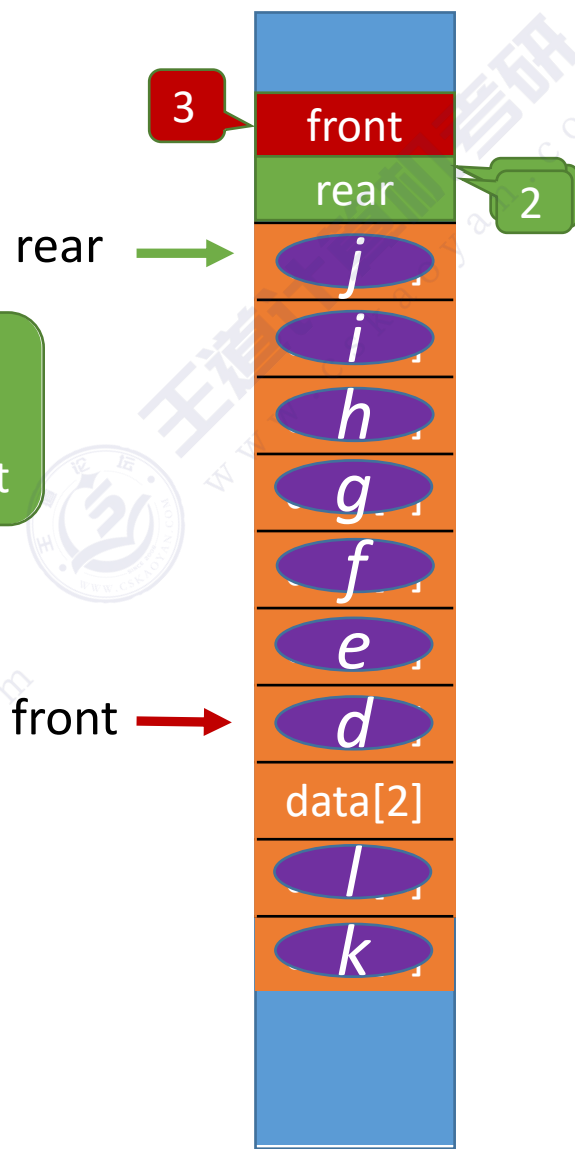
//队尾指针加1取模



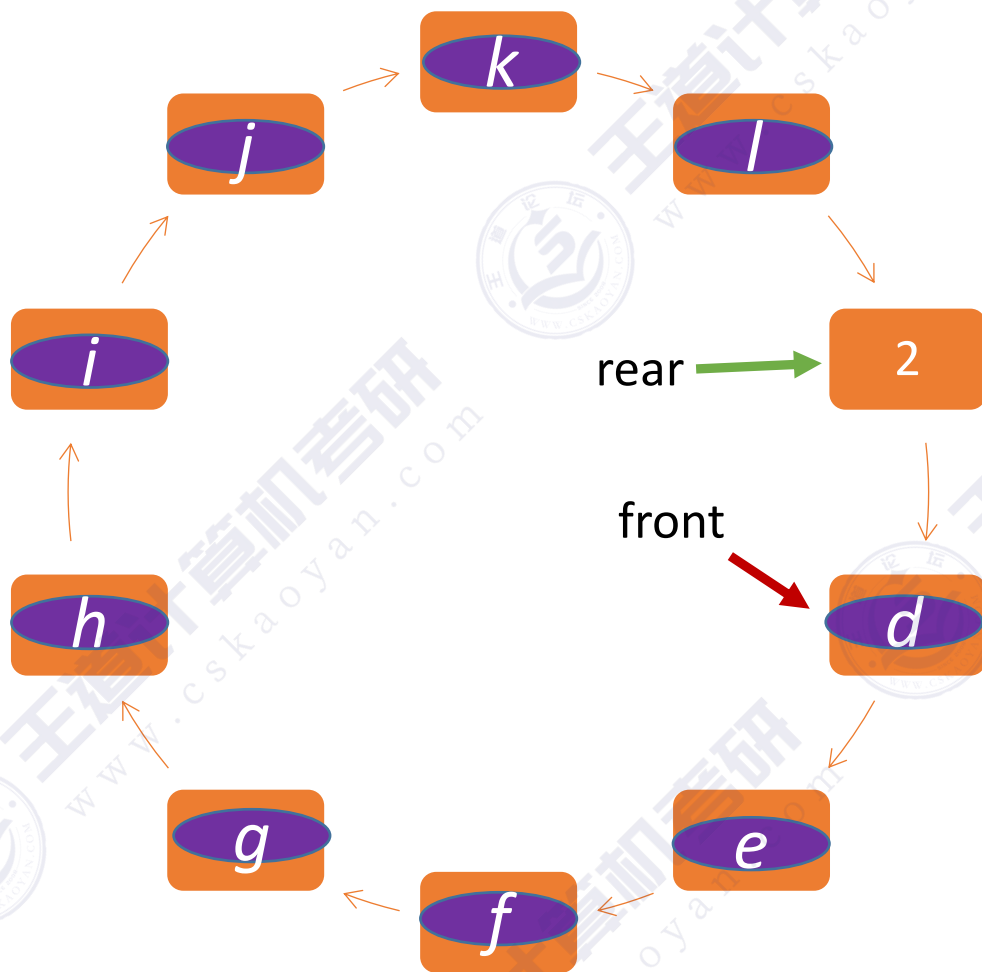
队列已满的条件：队尾指针的再下一个位置是队头，即 $(Q.rear+1)\%MaxSize==Q.front$

代价：牺牲一个存储单元

内存



循环队列——入队操作



//判断队列是否为空

```
bool QueueEmpty(SqQueue Q){
```

```
    if(Q.rear==Q.front) //队空条件
```

```
        return true;
```

```
    else
```

```
        return false;
```

```
}
```

//入队

```
bool EnQueue(SqQueue &Q, ElemType x){
```

```
    if((Q.rear+1)%MaxSize==Q.front)
```

```
        return false;
```

```
    Q.data[Q.rear]=x;
```

```
    Q.rear=(Q.rear+1)%MaxSize;
```

```
    return true;
```

```
}
```

判断
队空

判断
队满

//队满则报错

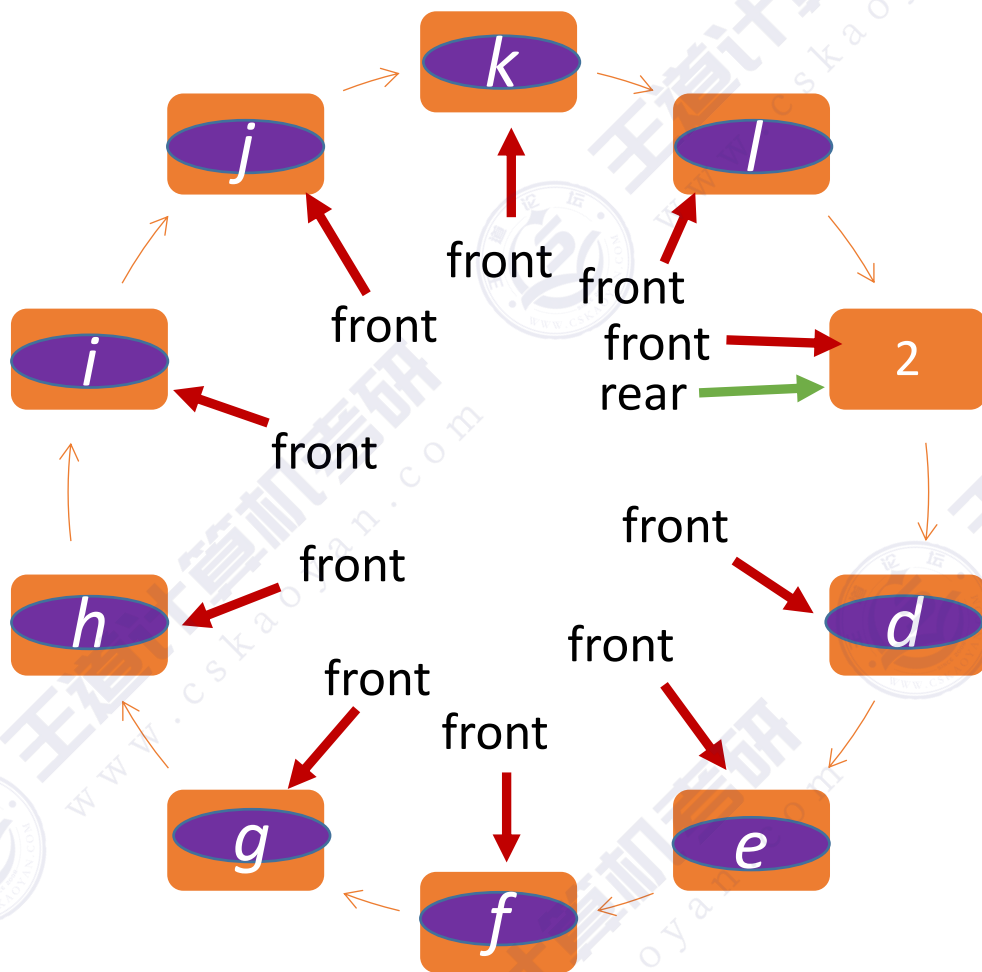
//新元素插入队尾

//队尾指针加1取模

用模运算将存储空间在
逻辑上变成了“环状”

循环队列——出队操作

只能让队头元素出队



//出队 (删除一个队头元素, 并用x返回)

```
bool DeQueue(SqQueue &Q, ElemType &x){
```

```
    if(Q.rear==Q.front)
```

判断队空

```
        return false; //队空则报错
```

```
    x=Q.data[Q.front];
```

```
    Q.front=(Q.front+1)%MaxSize;
```

```
    return true;
```

队头指针后移

```
}
```

//获得队头元素的值, 用x返回

```
bool GetHead(SqQueue Q, ElemType &x){
```

```
    if(Q.rear==Q.front)
```

```
        return false; //队空则报错
```

```
    x=Q.data[Q.front];
```

```
    return true;
```

```
}
```

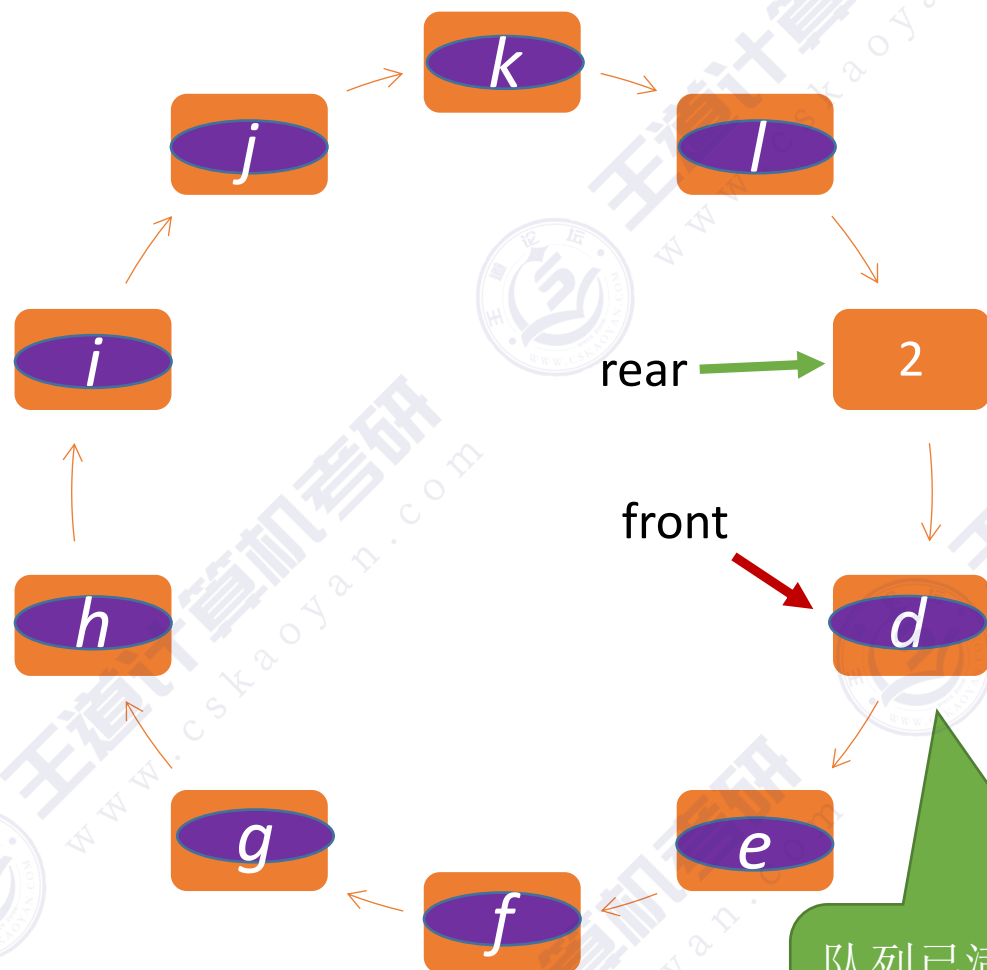
队列元素个数：
 $(\text{rear} + \text{MaxSize} - \text{front}) \% \text{MaxSize}$

方案一：判断队列已满/已空

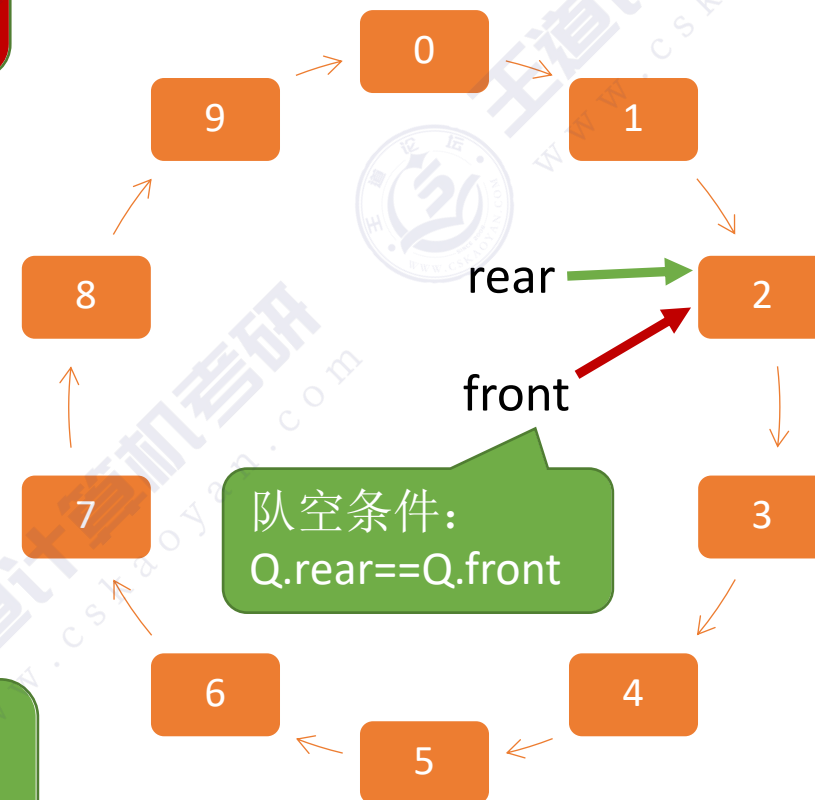
```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int front, rear;
} SqQueue;
```

初始化时
 $\text{rear} = \text{front} = 0$

出题老师：不准
浪费这可怜孤独的
存储空间！！！！



队列已满的条件：队尾指针的再下一个位置是队头，即
 $(\text{Q.rear} + 1) \% \text{MaxSize} == \text{Q.front}$



队空条件：
 $\text{Q.rear} == \text{Q.front}$

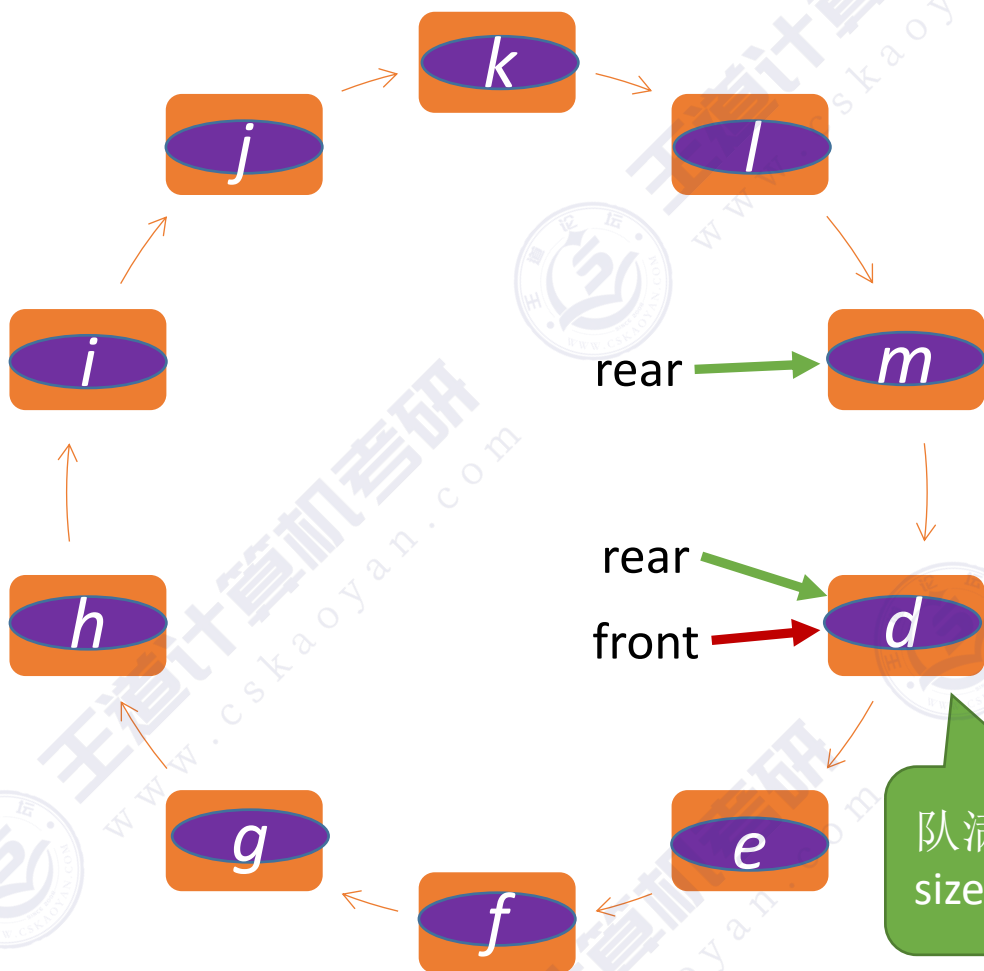
队列元素个数 = size

方案二：判断队列已满/已空

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int front, rear;
    int size; // 队列当前长度
} SqQueue;
```

插入成功 size++;
删除成功 size--;

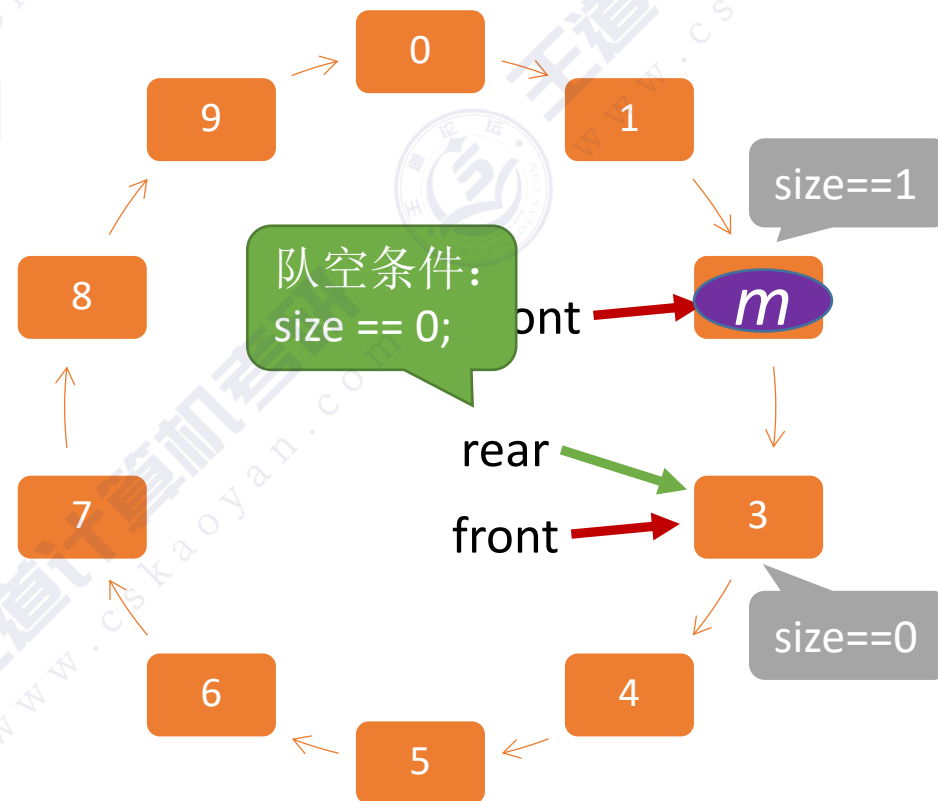
初始化时
rear=front=0;
size = 0;



rear → m size==MaxSize-1

rear → d front → d size==MaxSize

队满条件:
size==MaxSize



队空条件:
size == 0;

front → m

rear → 3

front → 3

size==0

队列元素个数:

$(rear + \text{MaxSize} - \text{front}) \% \text{MaxSize}$



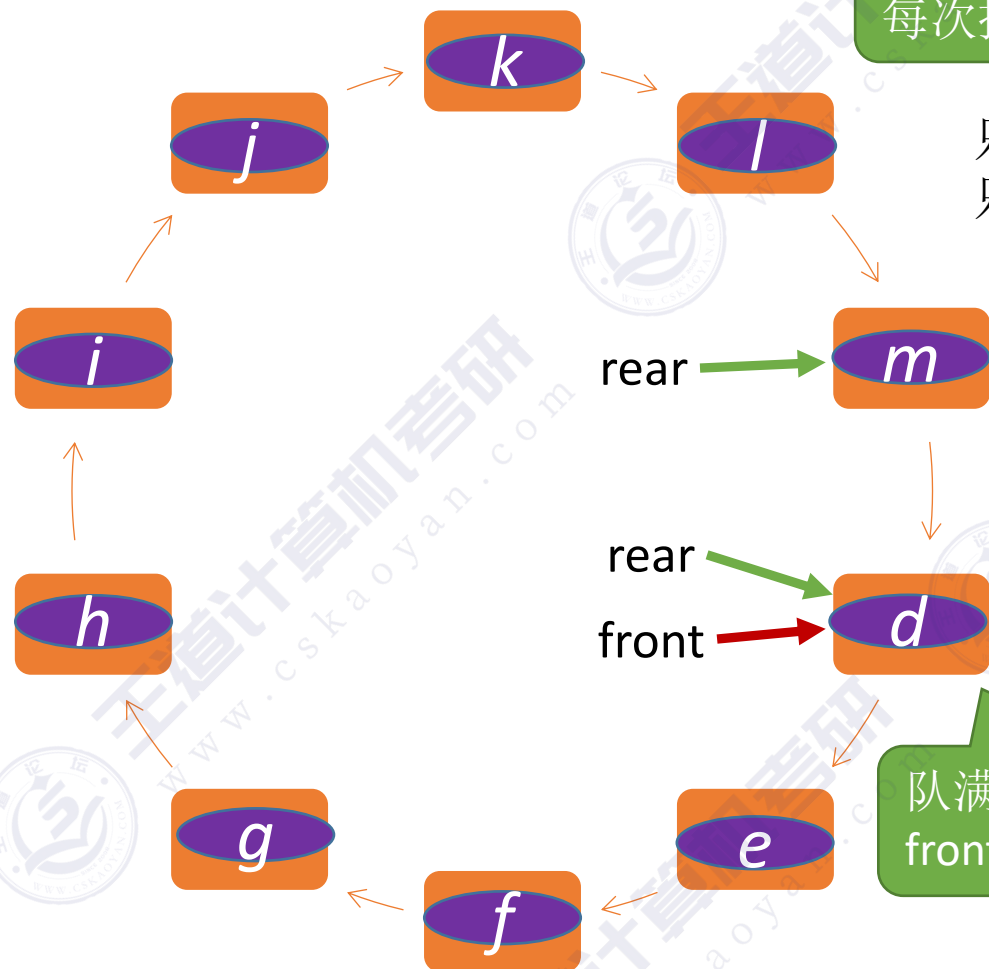
方案三：判断队列已满/已空

每次删除操作成功时，都令 $\text{tag} = 0$;
每次插入操作成功时，都令 $\text{tag} = 1$;

只有删除操作，才可能导致队空
只有插入操作，才可能导致队满

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int front, rear;
    int tag; //最近进行的是删除/插入
} SqQueue;
```

初始化时
 $\text{rear} = \text{front} = 0$;
 $\text{tag} = 0$;



rear

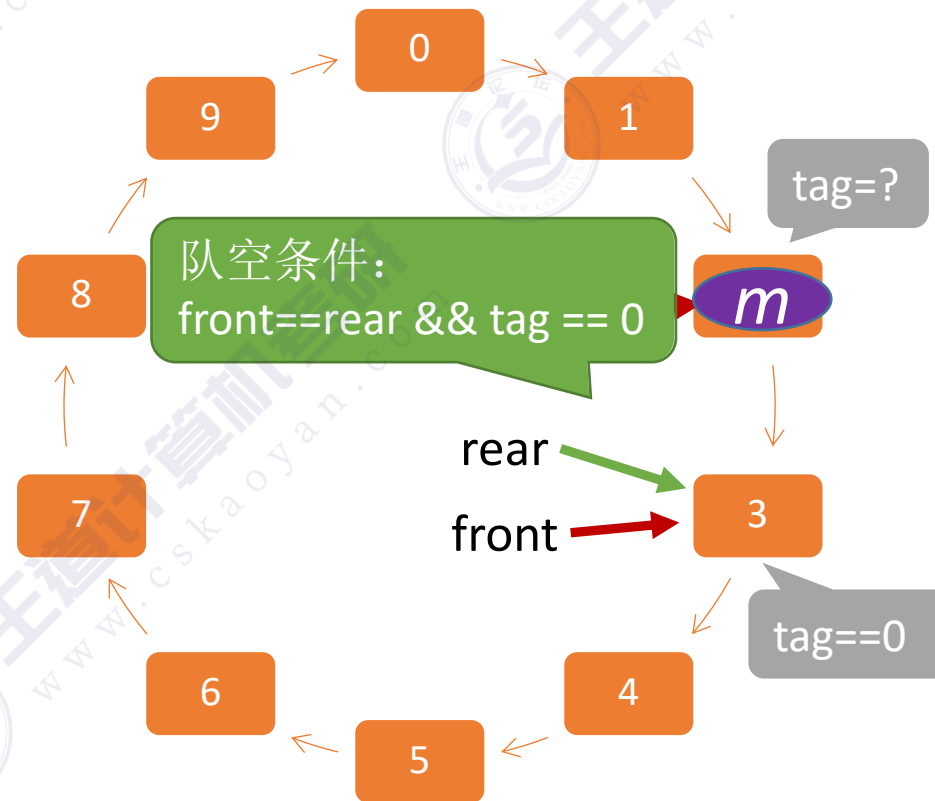
rear

front

tag==?

tag==1

队满条件:
 $\text{front} == \text{rear} \ \&\& \ \text{tag} == 1$

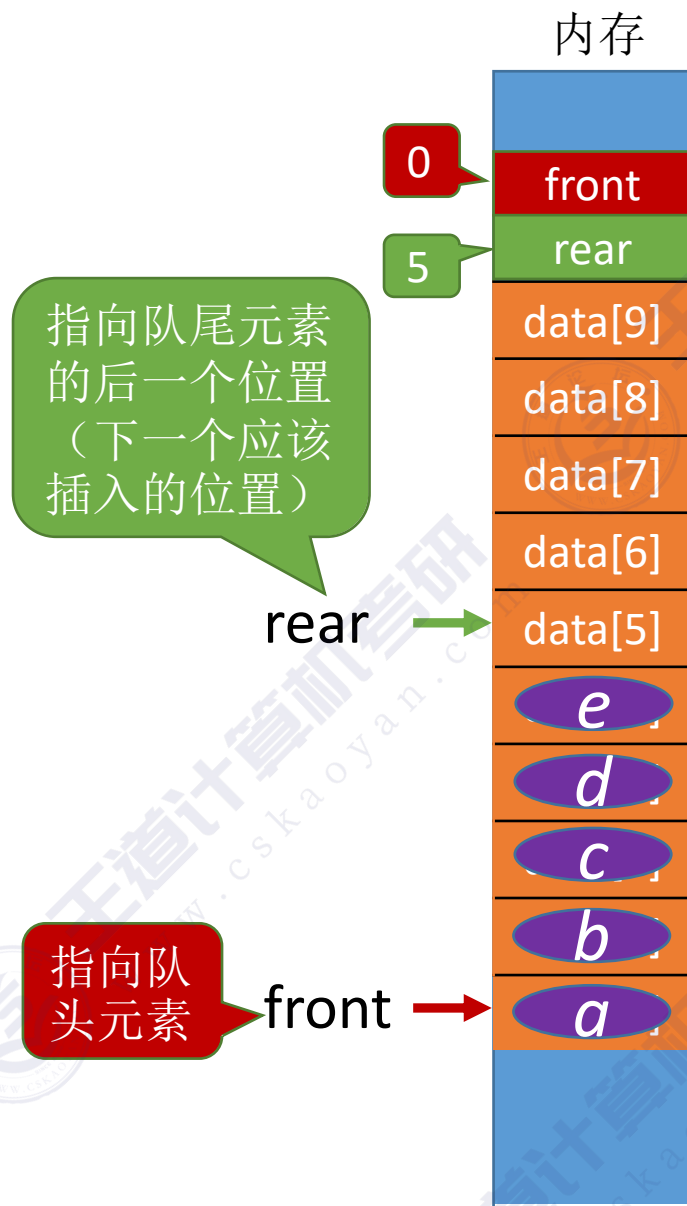


队空条件:
 $\text{front} == \text{rear} \ \&\& \ \text{tag} == 0$

tag=?

tag==0

其他出题方法



入队操作:

```
Q.data[Q.rear]=x;  
Q.rear=(Q.rear+1)%MaxSize;
```

入队导致
rear 后移

出队操作

```
x=Q.data[Q.front];  
Q.front=(Q.front+1)%MaxSize;
```

初始时队尾指针指向哪更合理?

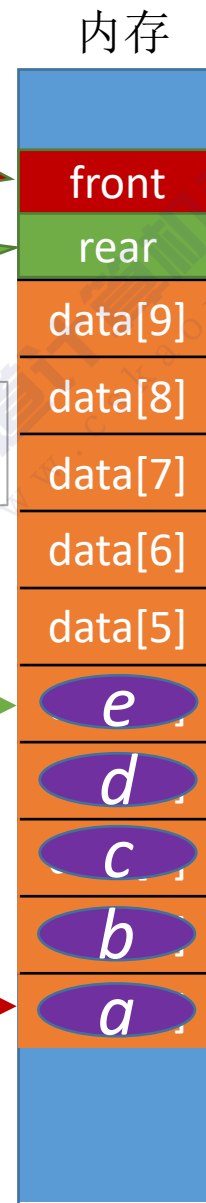
```
Q.rear=(Q.rear+1)%MaxSize;  
Q.data[Q.rear]=x;
```

指向队尾元素

rear

front

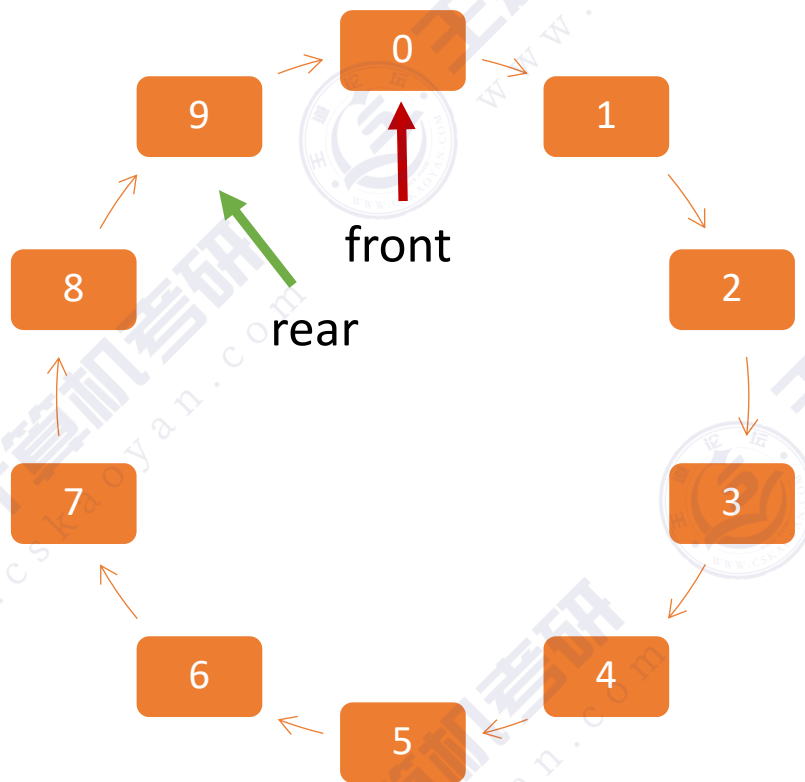
指向队头元素



其他出题方法

判空:

$(Q.rear+1)\%MaxSize==Q.front$



初始时队尾指针指向哪更合理?

```
Q.rear=(Q.rear+1)%MaxSize;  
Q.data[Q.rear]=x;
```

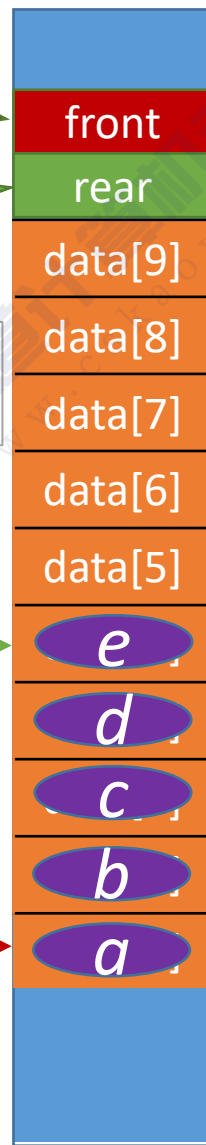
指向队尾元素

rear

front

指向队头元素

内存

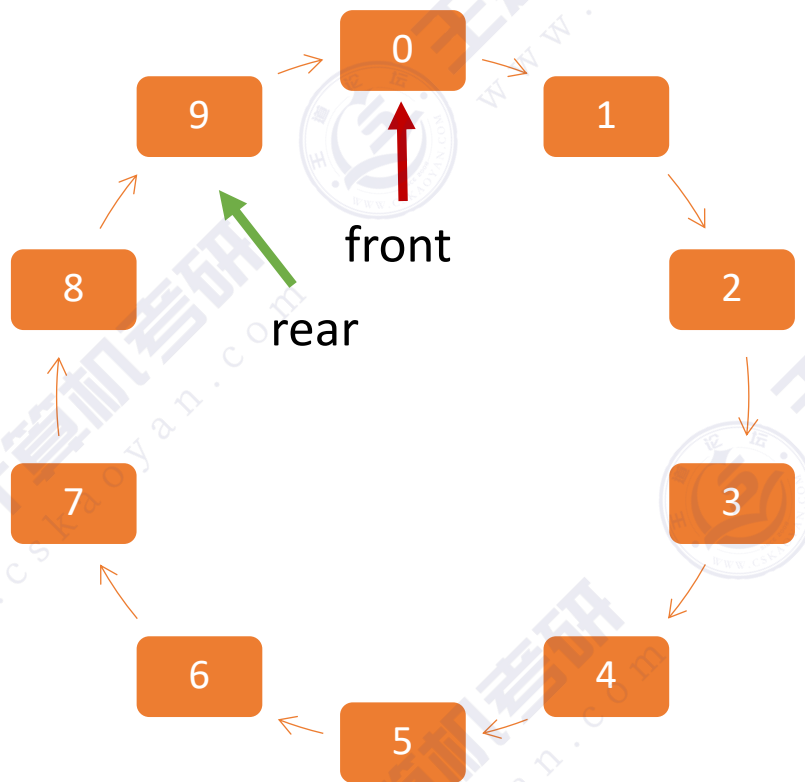


其他出题方法

方案一：牺牲一个存储单元
方案二：增加辅助变量

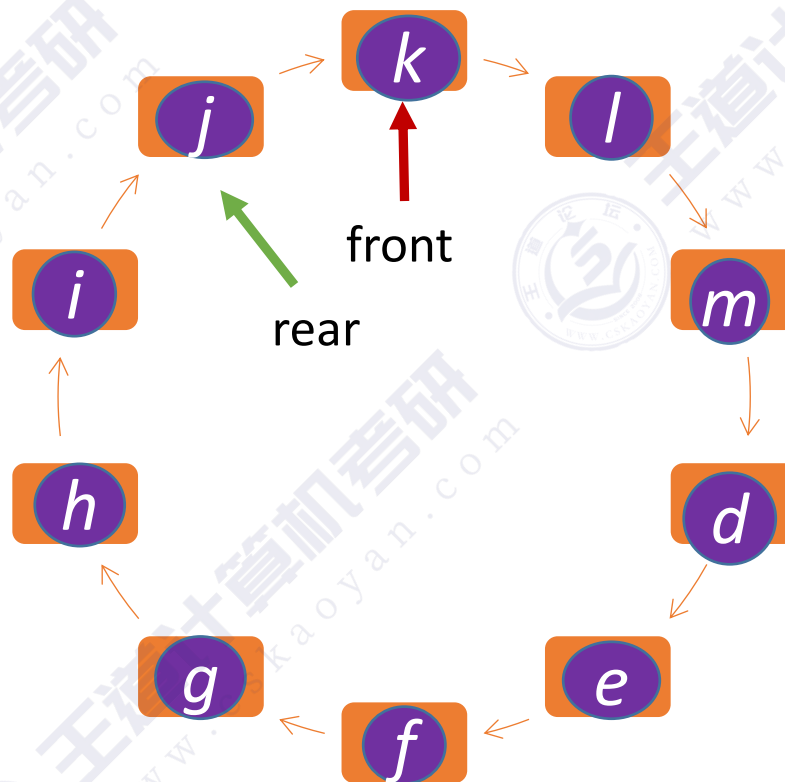
判空：

$(Q.rear+1)\%MaxSize==Q.front$



判满：

$(Q.rear+1)\%MaxSize==Q.front?$



知识回顾与重要考点

队列的顺序实现

实现思想

用静态数组存放数据元素，设置队头/队尾(front/rear)指针

循环队列：用模运算（取余）将存储空间在逻辑上变为“环状”

$Q.rear = (Q.rear + 1) \% \text{MaxSize};$

重要考点

如何 初始化、入队、出队

如何 判空、判满

如何计算队列的长度

思考：分别采用

①a、①b、①c

②a、②b、②c

策略时，这些操作怎么实现？

分析思路

确定front、rear指针的指向

①rear指向队尾元素后一个位置

②rear指向队尾元素

确定判空
判满的方法

a. 牺牲一个存储单元

b. 增加 size 变量记录队列长度

c. 增加 tag = 0/1 用于标记
最近的一次操作是 出队/入队

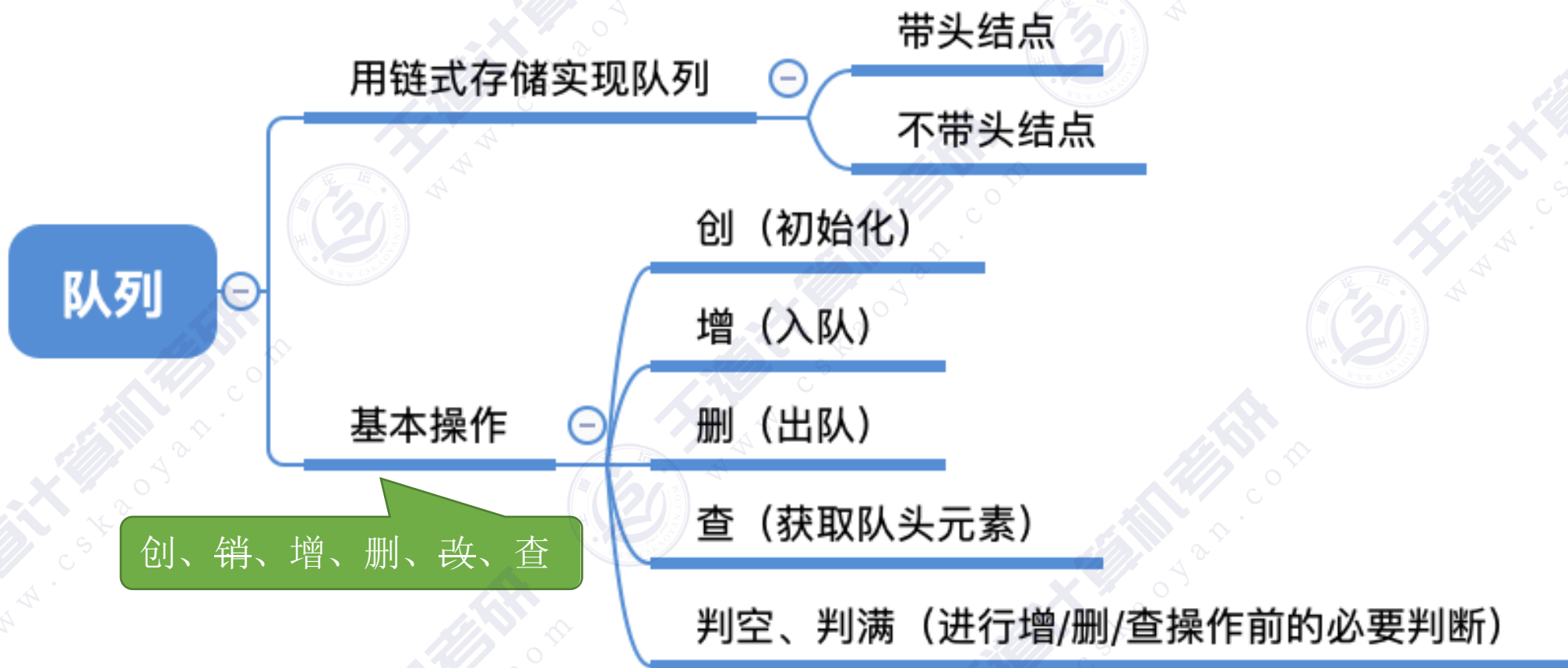
...

本节内容

队列

链式实现

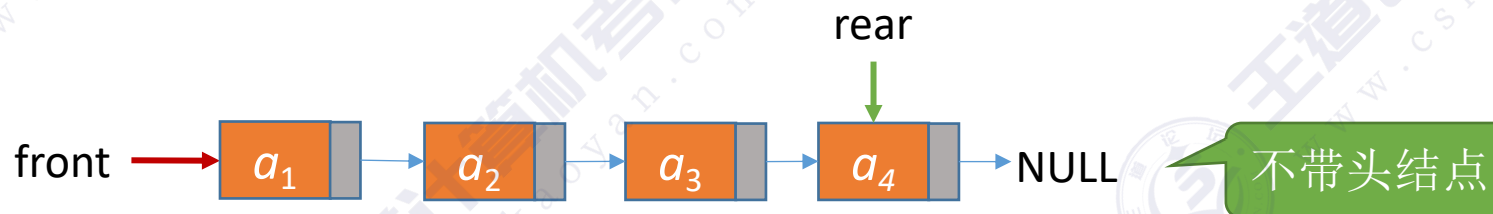
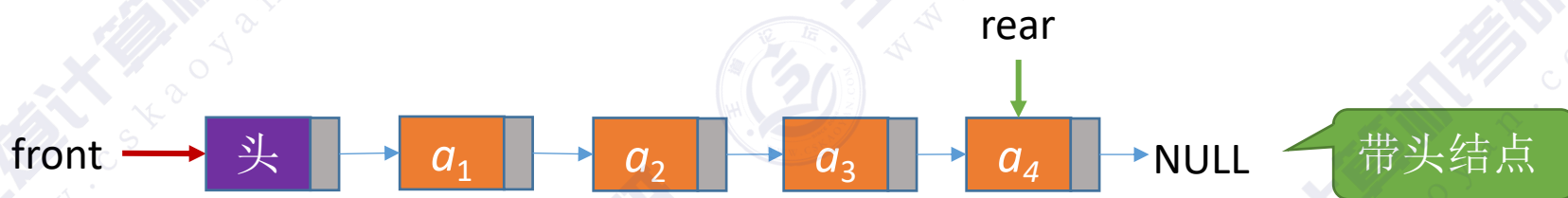
知识总览



队列的链式实现

```
typedef struct LinkNode{ //链式队列结点
    ElemType data;
    struct LinkNode *next;
}LinkNode;
```

```
typedef struct{ //链式队列
    LinkNode *front,*rear; //队列的队头和队尾指针
}LinkQueue;
```



链队列——链式存储实现的队列

初始化（带头结点）

```
typedef struct LinkNode{
    ElemType data;
    struct LinkNode *next;
}LinkNode;
```

//初始化队列(带头结点)

```
void InitQueue(LinkQueue &Q){
```

//初始时 front、rear 都指向头结点

```
    Q.front=Q.rear=(LinkNode*)malloc(sizeof(LinkNode));
```

```
    Q.front->next=NULL;
```

```
}
```

```
void testLinkQueue(){
```

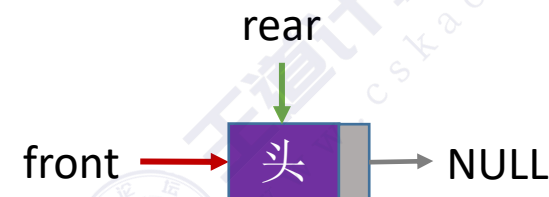
```
    LinkQueue Q; //声明一个队列
```

```
    InitQueue(Q); //初始化队列
```

//...后续操作...

```
}
```

```
typedef struct{
    LinkNode *front,*rear;
}LinkQueue;
```



带头结点空队列

//判断队列是否为空

```
bool IsEmpty(LinkQueue Q){
```

```
    if(Q.front==Q.rear)
```

```
        return true;
```

```
    else
```

```
        return false;
```

```
}
```

初始化（不带头结点）

//初始化队列(不带头结点)

```
void InitQueue(LinkQueue &Q){  
    //初始时 front、rear 都指向NULL  
    → Q.front=NULL;  
    → Q.rear=NULL;  
}
```

rear → NULL

front → NULL

不带头结点的空队列

//判断队列是否为空（不带头结点）

```
bool IsEmpty(LinkQueue Q){  
    if(Q.front==NULL)  
        return true;  
    else  
        return false;  
}
```

入队（带头结点）

//新元素入队（带头结点）

```
void EnQueue(LinkQueue &Q, ElemType x){
```

```
    LinkNode *s=(LinkNode *)malloc(sizeof(LinkNode));
```

```
    s->data=x;
```

```
    s->next=NULL;
```

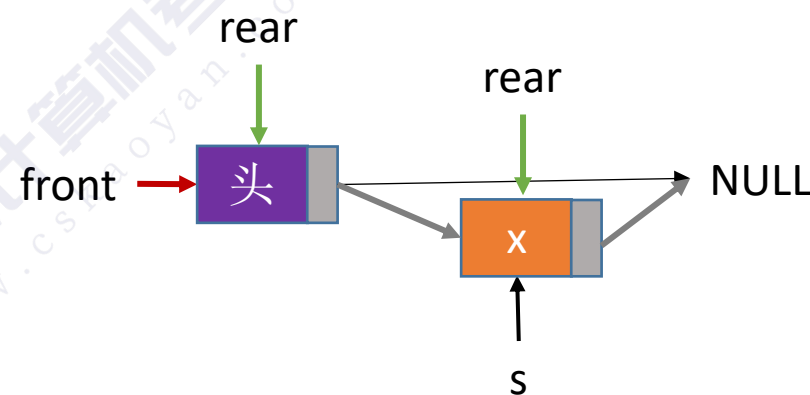
```
    Q.rear->next=s;
```

//新结点插入到rear之后

```
    Q.rear=s;
```

//修改表尾指针

```
}
```



入队（不带头结点）

//新元素入队（不带头结点）

```
void EnQueue(LinkQueue &Q, ElemType x){  
    LinkNode *s=(LinkNode *)malloc(sizeof(LinkNode));  
    s->data=x;  
    s->next=NULL;  
    if (Q.front == NULL){  
        Q.front = s;  
        Q.rear=s;  
    } else {  
        Q.rear->next=s;  
        Q.rear=s;  
    }  
}
```

//在空队列中插入第一个元素
//修改队头队尾指针

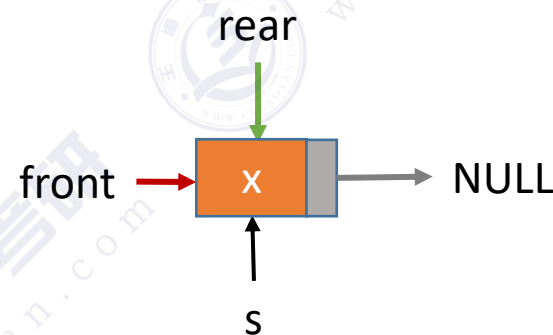
不带头结点的队列，第一个元素入队时需要特别处理

//新结点插入到 rear 结点之后
//修改 rear 指针

rear → NULL

front → NULL

不带头结点的空队列



出队（带头结点）

//队头元素出队（不带头结点）

```
bool DeQueue(LinkQueue &Q, ElemType &x){
```

```
    if(Q.front==Q.rear)
```

```
        return false;    //空队
```

```
    LinkNode *p=Q.front->next;
```

```
    x=p->data;    //用变量x返回队头元素
```

```
    Q.front->next=p->next;    //修改头结点的 next 指针
```

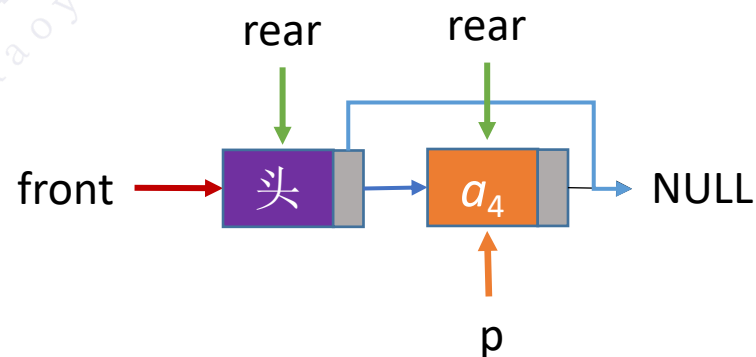
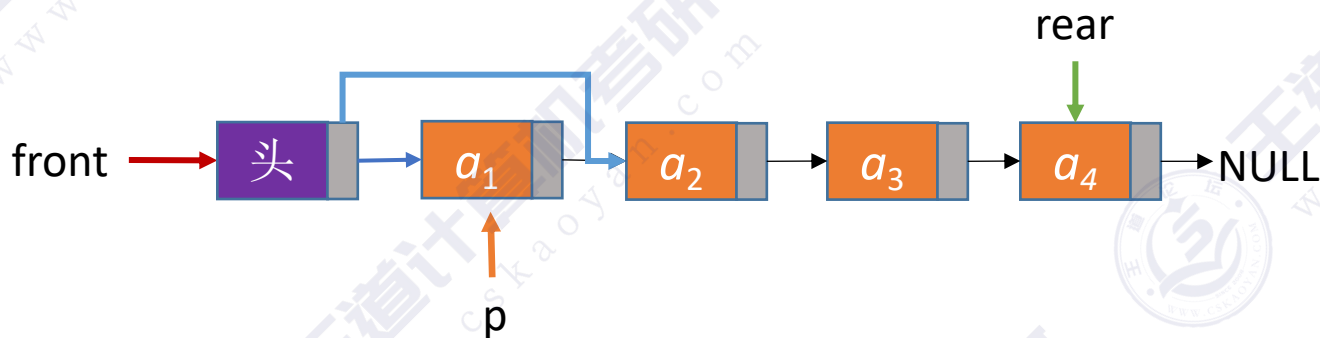
```
    if(Q.rear==p)    //此次是最后一个结点出队
```

```
        Q.rear=Q.front;    //修改 rear 指针
```

```
    free(p);    //释放结点空间
```

```
    return true;
```

```
}
```



出队（不带头结点）

//队头元素出队（不带头结点）

```
bool DeQueue(LinkQueue &Q, ElemType &x){
```

```
    if(Q.front==NULL)
```

```
        return false;
```

```
    LinkNode *p=Q.front;
```

```
    x=p->data;
```

```
    Q.front=p->next;
```

```
    if(Q.rear==p){
```

```
        Q.front = NULL;
```

```
        Q.rear = NULL;
```

```
    }
```

```
    free(p);
```

```
    return true;
```

```
}
```

//空队

//p指向此次出队的结点

//用变量x返回队头元素

//修改 front 指针

//此次是最后一个结点出队

//front 指向 NULL

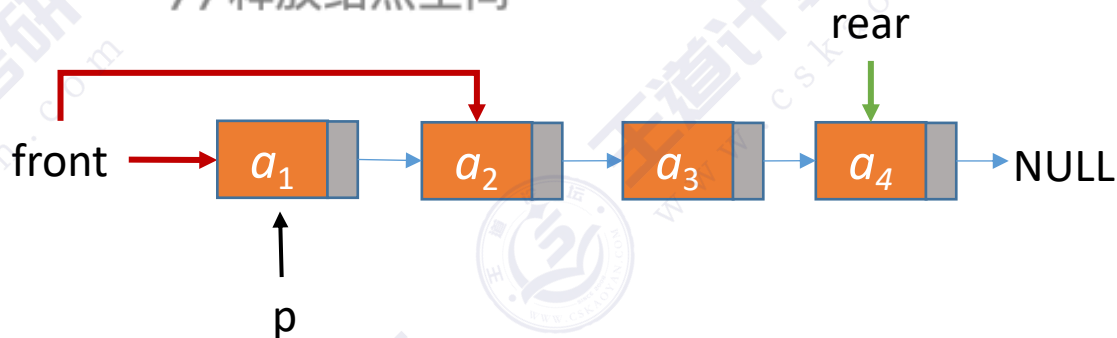
//rear 指向 NULL

//释放结点空间

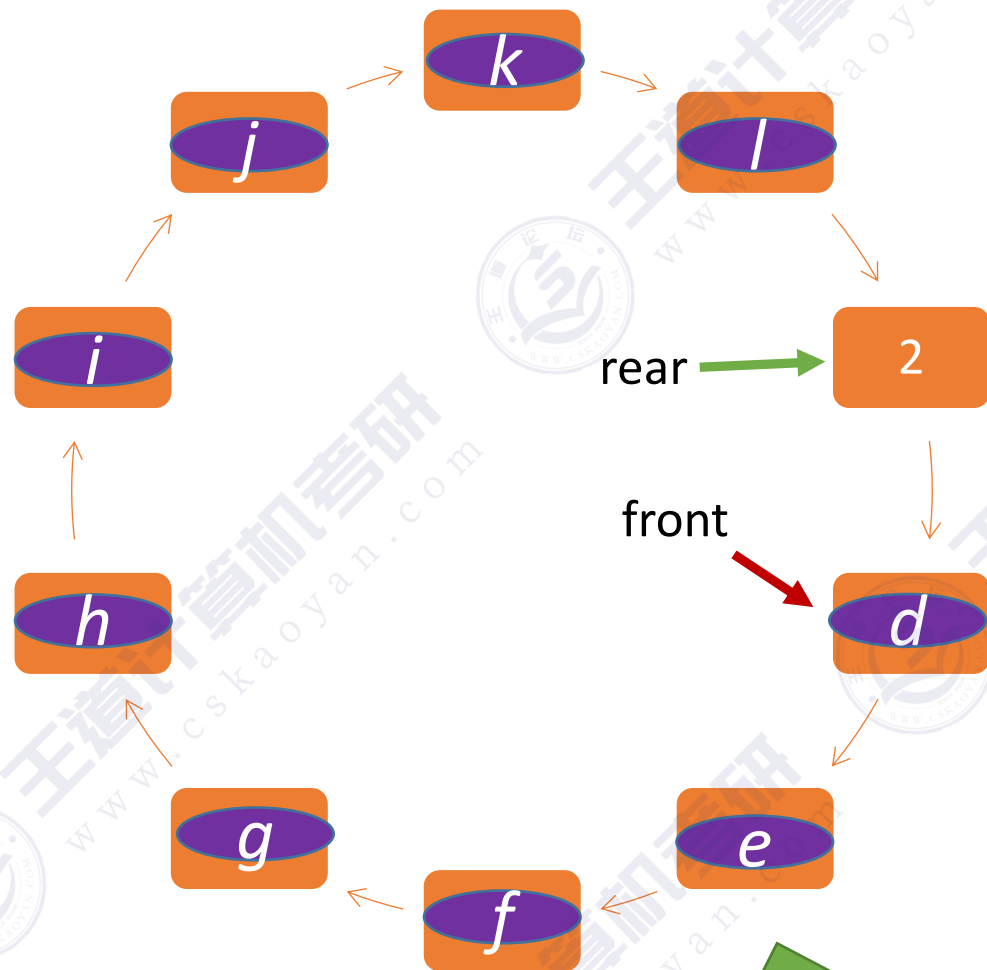
rear → NULL

front → NULL

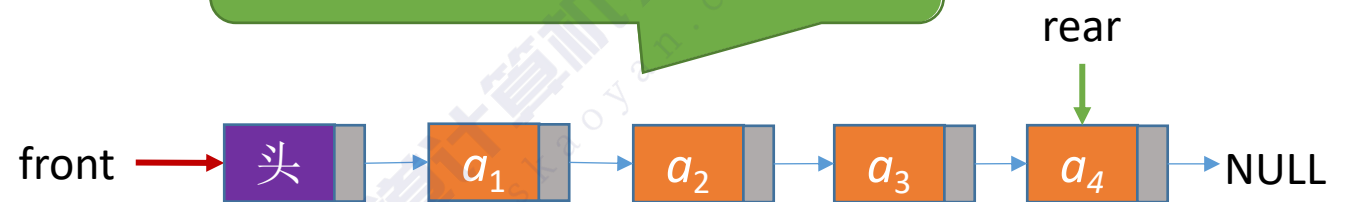
不带头结点的空队列



队列满的条件

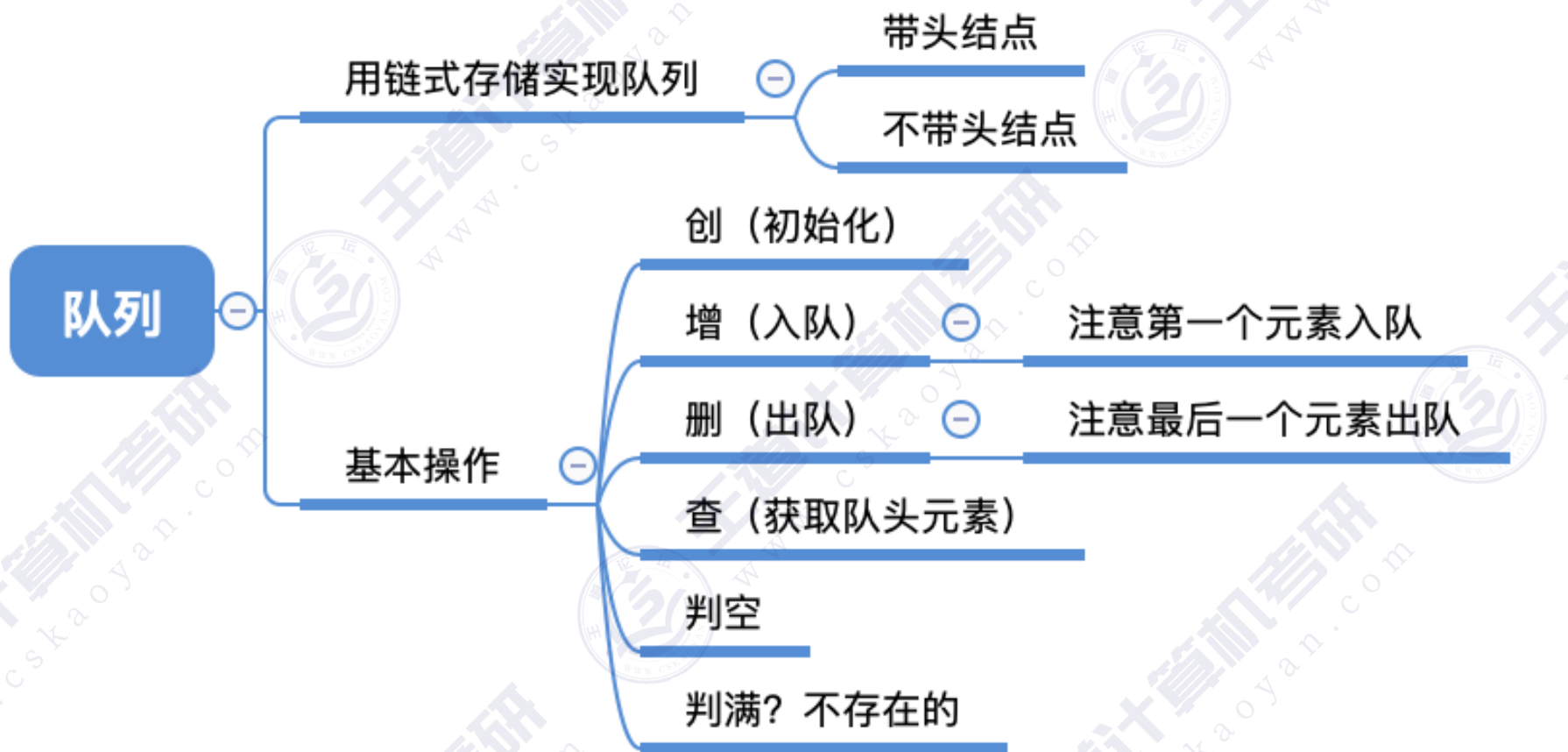


链式存储——一般不会队满，除非内存不足



顺序存储——预分配的空间耗尽时队满

知识回顾与重要考点



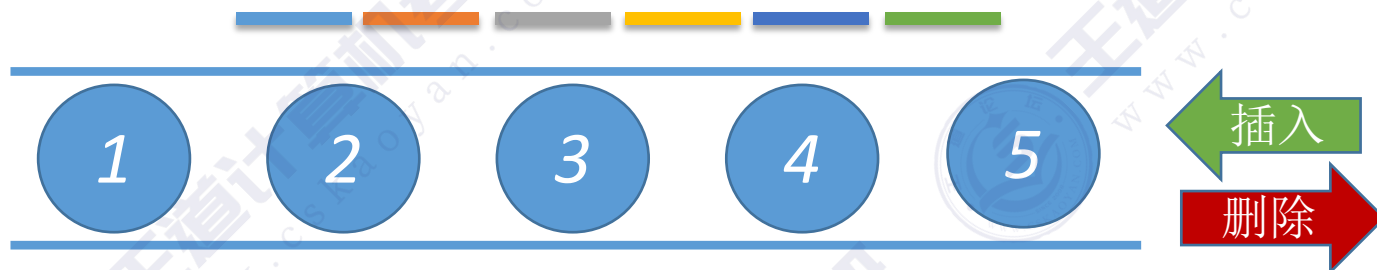
```
typedef struct LinkNode{
    ElemType data;
    struct LinkNode *next;
}LinkNode;
```

```
typedef struct{
    LinkNode *front,*rear;
}LinkQueue;
```


本节内容

双端队列

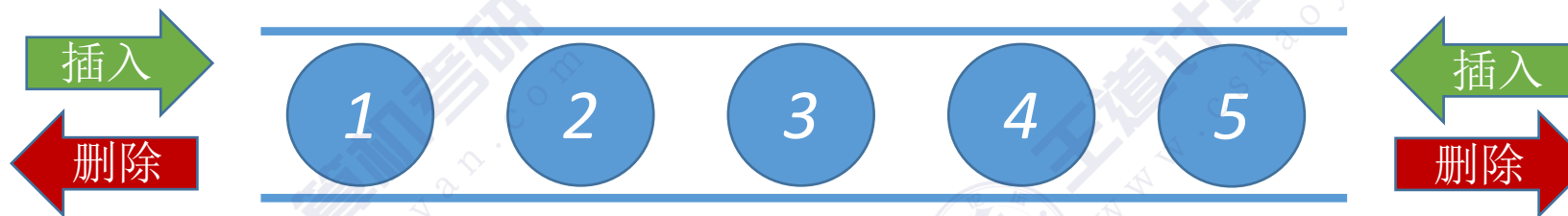
双端队列



栈：只允许从一端插入和删除的线性表



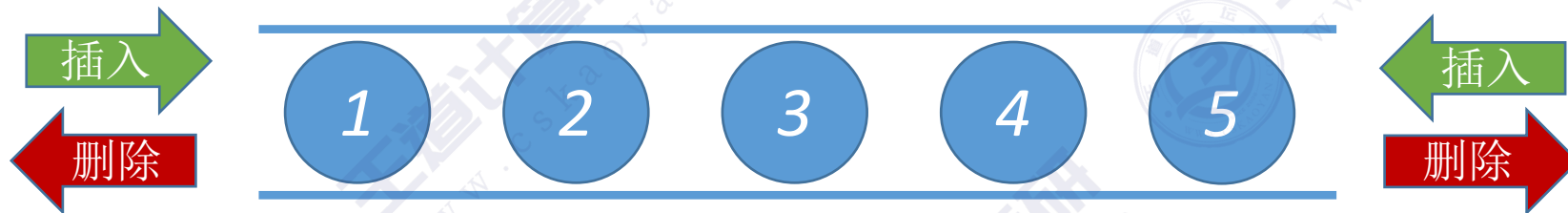
队列：只允许从一端插入、另一端删除的线性表



若只使用其中一端的插入、删除操作，则效果等同于栈

双端队列：只允许从两端插入、两端删除的线性表

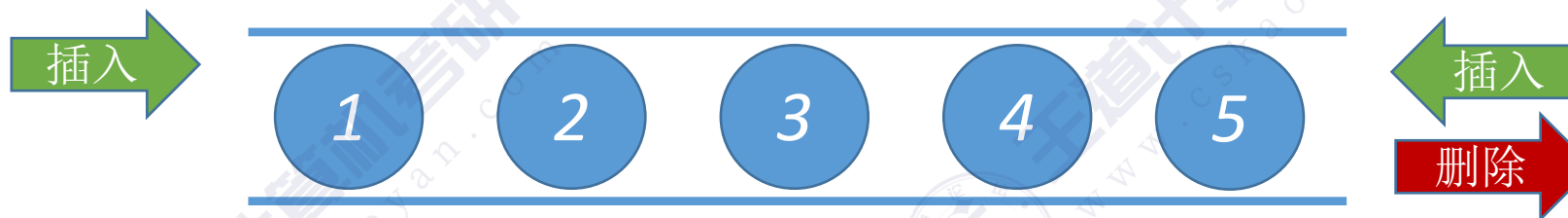
双端队列



双端队列：只允许从两端插入、两端删除的线性表



输入受限的双端队列：只允许从一端插入、两端删除的线性表

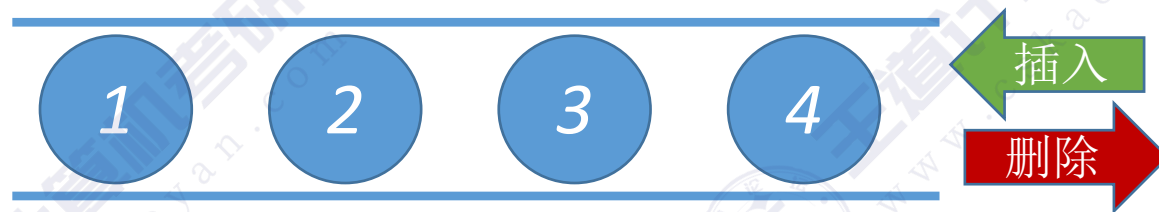


输出受限的双端队列：只允许从两端插入、一端删除的线性表

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$



栈

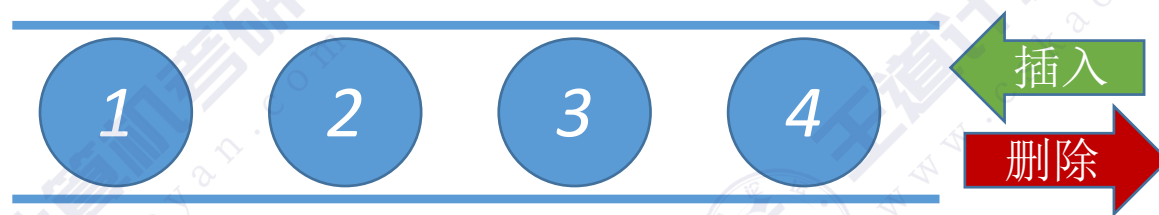
1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$

1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1



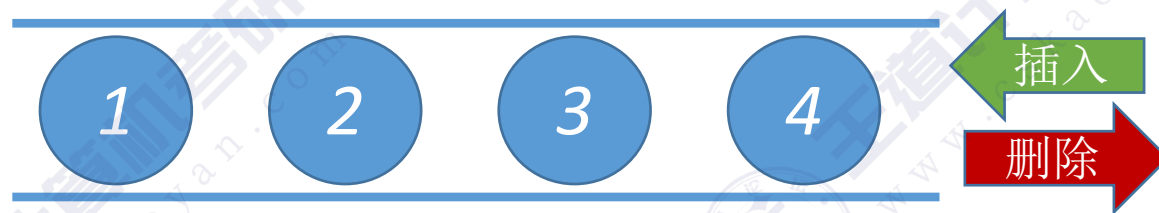
栈

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$

1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1



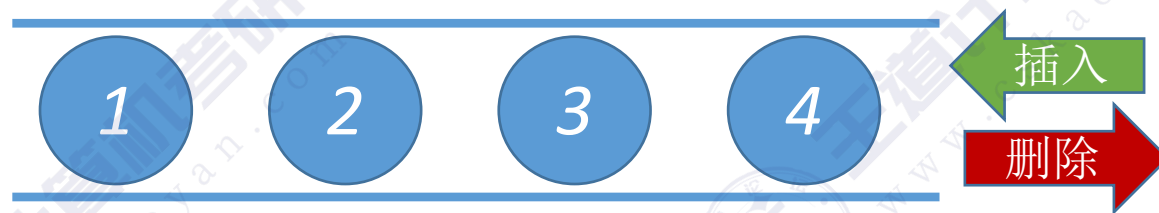
栈

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$

1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1



栈

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法？

$$A_4^4 = 4! = 24$$

1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1



栈

卡特兰数：

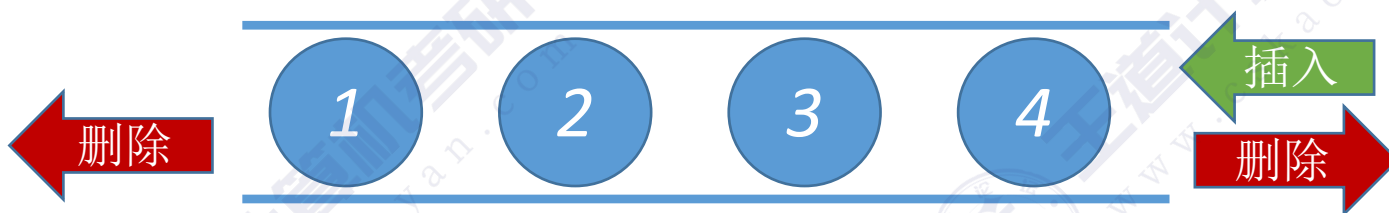
14种合法
出栈序列

$$\frac{1}{n+1} C_{2n}^n = \frac{1}{4+1} C_8^4 = 14$$

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法？

$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

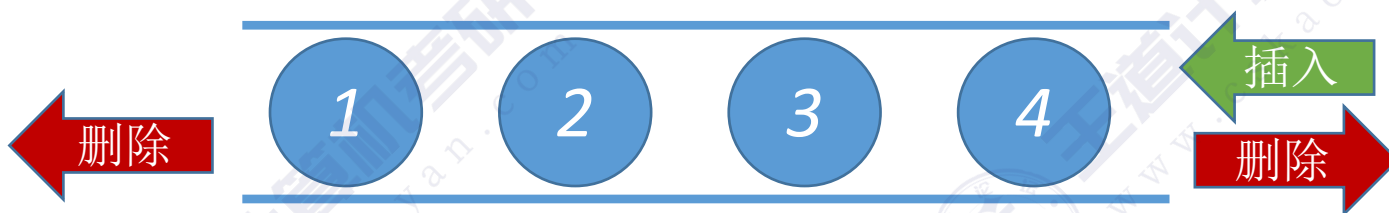
输入受限的双端队列

栈中合法的序列，双端队列中一定也合法

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法？

$$A_4^4 = 4! = 24$$



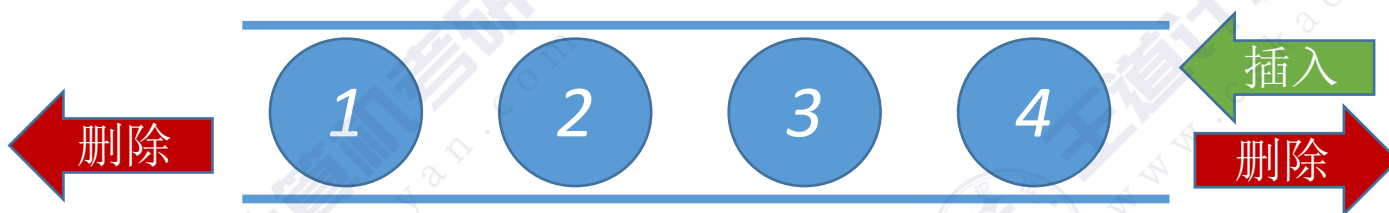
输入受限的双端队列

1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法？

$$A_4^4 = 4! = 24$$



输入受限的双端队列

1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法？

$$A_4^4 = 4! = 24$$

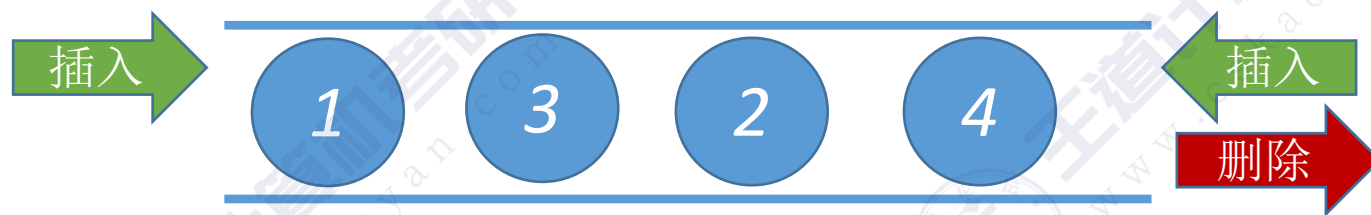
1,2,3,4	2,1,3,4	<u>3,1,2,4</u>	<u>4,1,2,3</u>
1,2,4,3	2,1,4,3	<u>3,1,4,2</u>	<u>4,1,3,2</u>
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
<u>1,4,2,3</u>	<u>2,4,1,3</u>	<u>3,4,1,2</u>	<u>4,3,1,2</u>
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输入受限的双端队列

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法？

$$A_4^4 = 4! = 24$$



输出受限的双端队列

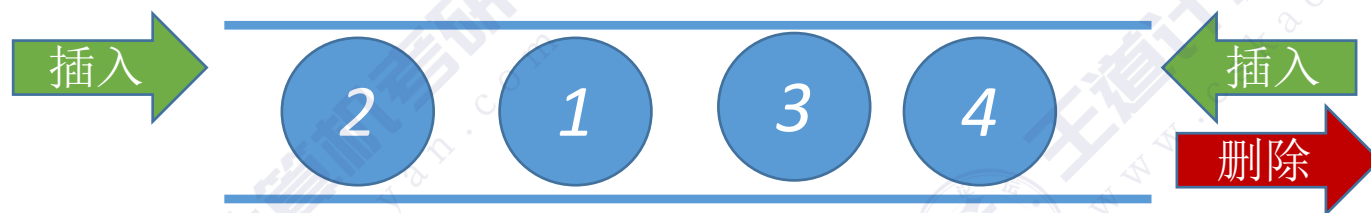
栈中合法的序列，双端队列中一定也合法

1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法？

$$A_4^4 = 4! = 24$$



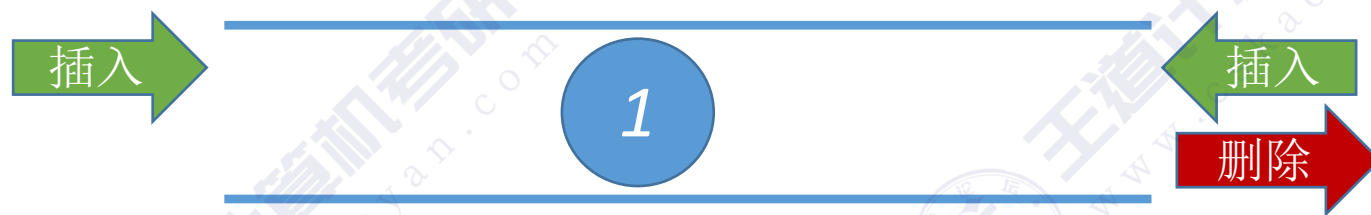
1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输出受限的双端队列

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法？

$$A_4^4 = 4! = 24$$



输出受限的双端队列

1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法？

$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	<u>3,1,2,4</u>	<u>4,1,2,3</u>
1,2,4,3	2,1,4,3	<u>3,1,4,2</u>	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	<u>4,2,1,3</u>
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
<u>1,4,2,3</u>	<u>2,4,1,3</u>	<u>3,4,1,2</u>	<u>4,3,1,2</u>
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输出受限的双端队列

知识回顾与重要考点

队列的变种

双端队列



允许从两端插入、两端删除的队列

输入受限的双端队列



允许从两端删除、从一端插入的队列

输出受限的双端队列



允许从两端插入、从一端删除的队列

考点：对输出序列合法性的判断

在栈中合法的输出序列，在双端队列中必定合法

回忆：栈的变种——共享栈